

Online Book Store SQL Project

1. Introduction

1.1 Project Overview

The Online Bookstore SQL Project is a practical simulation of a database system that forms the core of a modern e-commerce platform. It focuses on the entire data lifecycle, from designing a normalized database schema to performing complex analytical queries that drive business strategy. The project serves as a comprehensive demonstration of using SQL to manage and interpret data related to inventory, customer transactions, and overall sales performance.

1.2 Problem Statement & Objectives

In the highly competitive e-commerce landscape, leveraging data for decision-making is not just an advantage; it is a necessity. Businesses need to quickly and accurately answer questions about their customers, products, and operations to stay ahead.

The primary objective of this project is to use SQL to build an analytical framework capable of answering these fundamental business questions. This aligns with my core goal of "Converting raw data into valuable insights" by focusing on three key areas:

- **To design** a scalable and efficient relational database schema.
- **To demonstrate proficiency** in a wide range of SQL commands, from basic CRUD operations to advanced analytical functions.
- **To extract** actionable business intelligence regarding customer segmentation, sales trends, and inventory optimization.

1.3 Scope

The scope of this project includes the complete database design, creation of tables (**Books**, **Customers**, **Orders**), and the execution of 32 distinct analytical queries. The project focuses entirely on the back-end data layer and does not include the development of a front-end user interface or real-time transaction processing.

1.4 Technologies Used

- **Database:** PostgreSQL
- **Language:** SQL (DDL, DML, Joins, Aggregations, Window Functions)
- **Version Control:** Git & GitHub

2. Database Design and Schema

2.1 Conceptual Design

The database was designed using a relational model to ensure data integrity, scalability, and minimal data redundancy. Three core entities were identified as essential for the bookstore's operations:

- **Books:** To store all information related to product inventory.
- **Customers:** To maintain a record of all registered users.
- **Orders:** To act as a transactional link between customers and the books they purchase.

This three-table structure allows for a clear and logical separation of concerns while enabling powerful connections for analysis.

2.2 Logical Schema

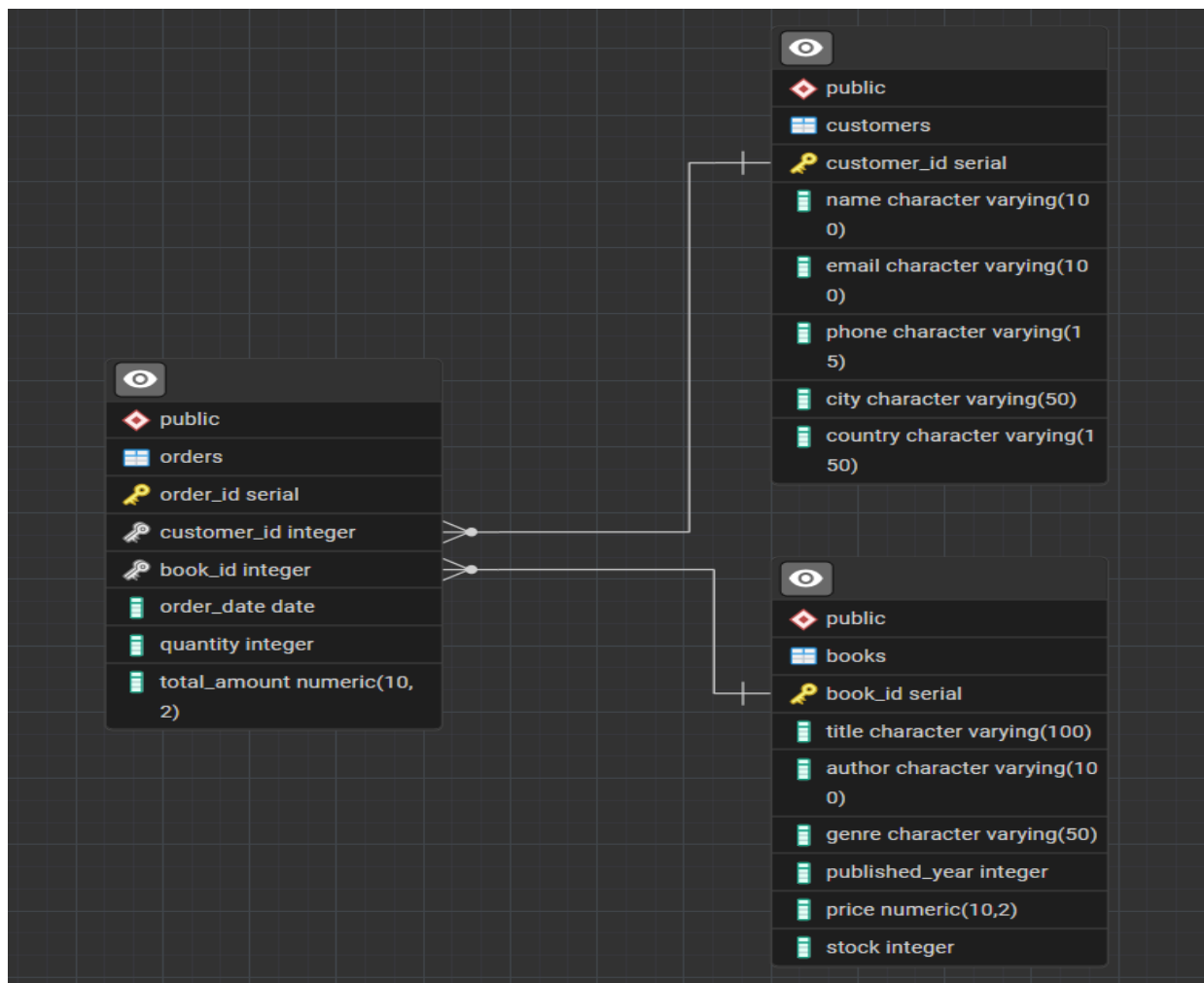
The logical schema consists of three tables with defined columns, data types, and constraints.

- **Table: Books**
 - **Description:** Stores all details for each book available in the inventory.
 - **Columns:**
 - **Book_ID:** SERIAL, PRIMARY KEY
 - **Title:** VARCHAR(100)
 - **Author:** VARCHAR(100)
 - **Genre:** VARCHAR(50)
 - **Published_Year:** INT
 - **Price:** NUMERIC(10, 2)
 - **Stock:** INT
- **Table: Customers**
 - **Description:** Stores information about each registered customer.
 - **Columns:**
 - **Customer_ID:** SERIAL, PRIMARY KEY
 - **Name:** VARCHAR(100)
 - **Email:** VARCHAR(100)
 - **Phone:** VARCHAR(15)
 - **City:** VARCHAR(50)
 - **Country:** VARCHAR(150)
- **Table: Orders**
 - **Description:** A transactional table linking customers to the books they ordered.
 - **Columns:**
 - **Order_ID:** SERIAL, PRIMARY KEY

- **Customer_ID**: INT, FOREIGN KEY (REFERENCES **Customers**)
- **Book_ID**: INT, FOREIGN KEY (REFERENCES **Books**)
- **Order_Date**: DATE
- **Quantity**: INT
- **Total_Amount**: NUMERIC(10, 2)

2.3 Schema Diagram

The following diagram illustrates the relationships between the tables. The **Orders** table connects **Customers** and **Books** in a many-to-many relationship (via the one-to-many links), forming the core of the database structure.



3. SQL Implementation and Analysis

This section presents a selection of the 32 queries written for this project, grouped by their business function. Each query is presented with its business question, the SQL code, and the strategic insight it provides.

3.1 Customer Analytics

Business Question (Q32): How can we rank our customers based on their total spending?

- **Approach:** This question requires aggregating total spending for each customer and then applying a rank. The **RANK()** window function is perfect for this, as it assigns a rank based on the **total_spent** value in descending order without needing complex self-joins or subqueries.

- **SQL Query:**

```
SELECT
    c.name,
    SUM(o.total_amount) AS total_spent,
    RANK() OVER (ORDER BY SUM(o.total_amount) DESC) AS spending_rank
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name;
```

- **Insight:** This query produces a prioritized list of high-value customers. This is critical intelligence for the marketing team, who can use it to create loyalty programs, offer exclusive discounts, and provide personalized service to retain their most valuable asset.

3.2 Sales and Revenue Performance

Business Question (Q25): What is the total revenue generated by each book genre?

- **Approach:** To determine revenue by genre, the **Orders** table (containing financial data) must be joined with the **Books** table (containing genre data). The **SUM()** function is then used to aggregate the **total_amount** for each **genre**.
- **SQL Query:**

```
SELECT
    b.genre,
    SUM(o.total_amount) AS total_revenue
```

```

FROM orders o
JOIN books b ON o.book_id = b.book_id
GROUP BY b.genre
ORDER BY total_revenue DESC;

```

- **Insight:** This analysis directly reveals which categories are the most profitable. The business can use this information to make strategic decisions about which genres to promote, which authors to feature, and where to focus their marketing budget.

3.3 Inventory and Product Management

Business Question (Q29): Which books are low in stock and may need to be reordered soon?

- **Approach:** This is a critical inventory management question. It requires calculating the current effective stock level by subtracting the total quantity sold from the initial stock. A **LEFT JOIN** is essential to include books that have never been sold. The **COALESCE** function handles **NULL** values gracefully, and the **HAVING** clause filters for books with a remaining stock of less than 5.
- **SQL Query**

```

SELECT
    b.title,
    b.stock - COALESCE(SUM(o.quantity), 0) AS stock_left
FROM books b
LEFT JOIN orders o ON b.book_id = o.book_id
GROUP BY b.book_id, b.title, b.stock
HAVING (b.stock - COALESCE(SUM(o.quantity), 0)) < 5;

```

- **Insight:** This query functions as a proactive alert system. It prevents "stockouts" of popular items, which can lead to lost sales and customer dissatisfaction. The inventory team can use this report daily to make timely reordering decisions.

4. Key Learnings and Challenges

- **Key Learnings:** This project provided deep, practical experience in advanced SQL. The implementation of **window functions** like `RANK()` proved invaluable for complex analysis that would otherwise be cumbersome. Using `LEFT JOIN` with `COALESCE` for inventory calculations was a key learning in handling potentially missing data (unsold books). Furthermore, designing a normalized schema from the ground up reinforced the fundamental principles of good database architecture.
- **Challenges Faced:** A primary challenge was ensuring the accuracy of complex aggregations, especially in the inventory management queries. Initially, using an `INNER JOIN` for the stock calculation query incorrectly excluded books that had never been sold. Overcoming this required a shift to a `LEFT JOIN` and careful handling of `NULL` values, which was a critical problem-solving step.

5. Conclusion and Future Scope

The Online Bookstore project successfully demonstrates the power of SQL as a comprehensive tool for both data management and business analysis. By designing a relational database and executing over 30 targeted queries, this project turned a simple transactional system into a source of strategic intelligence.

The insights generated can directly inform marketing strategy, inventory control, and sales forecasting. This project serves as a strong testament to the process of "Converting raw data into valuable insights."

Future Scope:

- **Automation:** Key reports, such as the low-stock alert, could be automated by implementing them as **Stored Procedures** that run on a schedule.
- **Performance Tuning:** As the database grows, **indexing** key columns (like `Genre`, `Author`, and foreign keys) would be crucial for maintaining query performance.
- **Integration:** The database could be integrated with a **Python/Flask** application to create a functional front-end for a complete full-stack project.

Basic Queries

Q1: Retrieve all books in the "Fiction" genre.

```
SQL
SELECT * FROM books
WHERE genre = 'Fiction';
```

Q2: Find books published after the year 1950.

```
SQL
SELECT * FROM books
WHERE published_year > 1950
ORDER BY published_year ASC;
```

Q3: List all customers from Canada.

```
SQL
SELECT * FROM customers
WHERE country = 'Canada';
```

Q4: Show orders placed in November 2023.

```
SQL
SELECT * FROM orders
WHERE EXTRACT(MONTH FROM order_date) = 11 AND EXTRACT(YEAR FROM
order_date) = 2023;
```

Q5: Retrieve the total stock of books available.

```
SQL
SELECT SUM(stock) AS total_stock
FROM books;
```

Q6: Find the details of the most expensive book.

```
SQL
SELECT * FROM books
ORDER BY price DESC
LIMIT 1;
```

Q7: Show all orders with a quantity greater than 1.

```
SQL
SELECT * FROM orders
WHERE quantity > 1
ORDER BY quantity DESC;
```

Q8: Retrieve all orders where the total amount exceeds \$20.

```
SQL
SELECT * FROM orders
WHERE total_amount > 20
ORDER BY total_amount DESC;
```

Q9: Find the book with the lowest stock.

```
SQL
SELECT * FROM books
ORDER BY stock ASC
LIMIT 1;
```

Q10: Calculate the total revenue generated from all orders.

```
SQL
SELECT SUM(total_amount) AS total_revenue
FROM orders;
```

Q11: Find the total number of books in stock, grouped by genre.

```
SQL
SELECT genre, SUM(stock) AS stock
FROM books
GROUP BY genre;
```

Intermediate to Advanced Queries

Q12: List the top 5 customers who ordered the most books.

```
SQL
SELECT
    c.Name,
    SUM(o.Quantity) AS Total_Books_Ordered
FROM Orders o
JOIN Customers c ON c.Customer_ID = o.Customer_ID
GROUP BY c.Customer_ID, c.Name
ORDER BY Total_Books_Ordered DESC
LIMIT 5;
```

Q13: Create a view to show best-selling books (more than 50 units sold).

```
SQL
CREATE VIEW BestSellers AS
SELECT
    B.Title,
    SUM(O.Quantity) AS Total_Sold
FROM Orders O
JOIN Books B ON O.Book_ID = B.Book_ID
GROUP BY B.Title
HAVING SUM(O.Quantity) > 50;
```

Q14: Retrieve the total number of books sold for each genre.

```
SQL
SELECT
    b.Genre,
    SUM(o.Quantity) AS Total_Sold_Books
FROM Orders o
JOIN Books b ON o.Book_ID = b.Book_ID
GROUP BY b.Genre
ORDER BY b.Genre;
```

Q15: Find the average price of books in the "Fantasy" genre.

```
SQL
SELECT
    genre,
    AVG(price) AS average_price
FROM books
WHERE genre = 'Fantasy'
GROUP BY genre;
```

Q16: List customers who have placed at least 2 orders.

```
SQL
SELECT
    c.name,
    COUNT(o.order_id) AS number_of_orders
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.name
HAVING COUNT(o.order_id) >= 2;
```

Q17: Find the most frequently ordered book.

```
SQL
SELECT
    b.title,
    COUNT(o.order_id) AS order_count
FROM orders o
JOIN books b ON o.book_id = b.book_id
GROUP BY b.title
ORDER BY order_count DESC
LIMIT 1;
```

Q18: Show the top 3 most expensive books in the 'Fantasy' Genre.

```
SQL
SELECT * FROM books
WHERE genre='Fantasy'
ORDER BY price DESC
LIMIT 3;
```

Q19: Retrieve the total quantity of books sold by each author.

```
SQL
SELECT
    b.author,
    SUM(o.quantity) AS books_sold
FROM books b
JOIN orders o ON b.book_id = o.book_id
GROUP BY b.author;
```

Q20: List the cities where customers who spent over \$200 are located.

```
SQL
SELECT
    c.city,
    SUM(o.total_amount) AS total_spent
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.city
HAVING SUM(o.total_amount) > 200;
```

Q21: Find the top 5 customers who spent the most on orders.

```
SQL
SELECT
    c.name,
    SUM(o.total_amount) as total_spent
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.name
ORDER BY total_spent DESC
LIMIT 5;
```

Q22: Calculate the stock remaining after fulfilling all orders.

```
SQL
SELECT
    b.book_id,
    b.title,
    b.stock - COALESCE(SUM(o.quantity), 0) AS stocks_left
FROM books b
LEFT JOIN orders o ON b.book_id = o.book_id
GROUP BY b.book_id, b.title, b.stock
ORDER BY stocks_left;
```

Q23: List books that have never been ordered.

```
SQL
SELECT
    b.book_id,
    b.title
FROM books b
```

```
LEFT JOIN orders o ON b.book_id = o.book_id
WHERE o.order_id IS NULL;
```

Q24: Retrieve the most recent order placed by each customer.

```
SQL
SELECT
    c.customer_id,
    c.name,
    MAX(o.order_date) AS last_order_date
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name;
```

Q25: Find the total revenue generated by each genre.

```
SQL
SELECT
    b.genre,
    SUM(o.total_amount) AS total_revenue
FROM orders o
JOIN books b ON o.book_id = b.book_id
GROUP BY b.genre
ORDER BY total_revenue DESC;
```

Q26: List customers who ordered books from more than one genre.

```
SQL
SELECT
    c.customer_id,
    c.name,
    COUNT(DISTINCT b.genre) AS genre_count
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN books b ON o.book_id = b.book_id
GROUP BY c.customer_id, c.name
HAVING COUNT(DISTINCT b.genre) > 1;
```

Q27: Find the average quantity ordered per book title.

```
SQL
SELECT
```

```
    b.title,  
    ROUND(AVG(o.quantity), 2) AS avg_quantity_ordered  
FROM books b  
JOIN orders o ON b.book_id = o.book_id  
GROUP BY b.title;
```

Q28: Get the month-wise total number of orders placed.

```
SQL  
SELECT  
    DATE_TRUNC('month', o.order_date)::DATE AS order_month,  
    COUNT(o.order_id) AS total_orders  
FROM orders o  
GROUP BY order_month  
ORDER BY order_month;
```

Q29: Which books are low in stock (less than 5 units left after orders)?

```
SQL  
SELECT  
    b.title,  
    b.stock - COALESCE(SUM(o.quantity), 0) AS stock_left  
FROM books b  
LEFT JOIN orders o ON b.book_id = o.book_id  
GROUP BY b.book_id, b.title, b.stock  
HAVING (b.stock - COALESCE(SUM(o.quantity), 0)) < 5;
```

Q30: For each customer, show the total books ordered and total amount spent.

```
SQL  
SELECT  
    c.name,  
    SUM(o.quantity) AS total_books_ordered,  
    SUM(o.total_amount) AS total_spent  
FROM customers c  
JOIN orders o ON c.customer_id = o.customer_id  
GROUP BY c.name;
```

Q31: Show genres where more than 100 books have been sold.

```
SQL  
SELECT
```

```
    b.genre,  
    SUM(o.quantity) AS total_sold  
FROM orders o  
JOIN books b ON o.book_ID = b.book_id  
GROUP BY b.genre  
HAVING SUM(o.quantity) > 100;
```

Q32: Rank customers based on total spending using RANK().

```
SQL  
SELECT  
    c.name,  
    SUM(o.total_amount) AS total_spent,  
    RANK() OVER (ORDER BY SUM(o.total_amount) DESC) AS spending_rank  
FROM customers c  
JOIN orders o ON c.customer_id = o.customer_id  
GROUP BY c.customer_id, c.name;
```

Thank you for your time